

Ginza入門書

1. この文書の目的

この文書は、`memory_project` における GiNZA 導入を検討するための入門書です。

対象読者は、現在のルールベース解析器を見て「GiNZA を入れると何が変わるのか」「なぜ他ライブラリではなく GiNZA を選ぶのか」を把握したい開発者です。

扱う内容は次の通りです。

- GiNZA とは何か
- 今の解析器が何をしていて、どこが弱い
- GiNZA の基本的な使い方
- `token`, `pos`, `lemma`, `ent`, `dependency` が何に使えるか
- 他ライブラリとの比較
- `memory_project` で GiNZA をどう導入するか

2. 現在の解析器は何をしているか

現在の解析器は `[app/analysis/text_analyzer.py](/Users/kabuto/Documents/Codex/memory_project/app/analysis/text_analyzer.py)` にあります。

ここでは主に次の材料を使っています。

- 正規表現 `WORD_RE`
- `EMOTION_KEYWORDS`
- `MEMORY_TYPE_RULES`
- `FUTURE_TERMS`
- `SENSITIVE_TERMS`
- `TECHNICAL_MARKERS`

流れとしては、

1. 入力文から正規表現で `token` らしい文字列を切り出す
2. `keyword` と `topic` を雑に作る
3. 文字列一致で `memory_types` と `emotion` を推定する
4. 手書きルールで `scores` と `save_strength` を計算する

5. `reason_codes`, `recall_policy`, `safety`, `facets` を組み立てる

という形です。

この方式は軽くて追いやすい一方で、次の弱さがあります。

- token の切り方が雑
- 活用語の原形を見ていない
- 固有名詞をちゃんと扱えない
- 否定に弱い
- 文のどこが主張なのかを見ていない
- `topic` と `keyword` が浅い

3. GiNZA とは何か

GiNZA は、spaCy 上で動く日本語 NLP ライブラリです。

日本語の token 化、品詞推定、原形化、係り受け解析、固有表現抽出などを、spaCy の API で扱えるようにします。

`memory_project` の文脈で言うと、GiNZA は「感情や記憶を直接理解してくれる魔法の箱」ではありません。

正しくは、「日本語の文をルールで扱いやすい形へ分解してくれる土台」です。

この土台があると、

- 名詞中心の `topic` 抽出
- 原形ベースの `keyword` 抽出
- 固有表現の分離
- 否定の検出
- `reason_codes` の根拠づけ

がしやすくなります。

4. GiNZA の基本的な使い方

最小の使い方は次の通りです。

```
import spacy

nlp = spacy.load("ja_ginza")
```

```
doc = nlp("本屋で自分で本を選ぶのが面倒で、AIに一冊決めてほしい。")
```

この時点で `doc` に、日本語解析結果が入っています。

4.1 token

```
[token.text for token in doc]
```

`token` は文を分割した最小単位です。

今の `WORD_RE.findall(...)` より自然な単位で扱えます。

4.2 pos

```
[(token.text, token.pos_) for token in doc]
```

`pos_` は品詞です。例えば `NOUN`, `PROPN`, `VERB`, `ADJ` などが入ります。

`topic` を名詞中心で取る、感情表現を形容詞・動詞から取る、といった改善に使えます。

4.3 lemma

```
[(token.text, token.lemma_) for token in doc]
```

`lemma_` は原形です。

例えば「感じた」「感じる」「感じている」を同じ語として扱いやすくなります。

今の `MEMORY_TYPE_RULES`

は表層文字列一致なので、「活用が変わると取りこぼす」弱さがあります。GINZA を入れるとここを改善できます。

4.4 ent

```
[(ent.text, ent.label_) for ent in doc.ents]
```

`ent` は固有表現です。

人名、組織名、場所名、作品名などを抽出できる場合があります。

`memory_project` では、関係性の記憶、場所、出来事、検索性タグ生成に使えます。

4.5 dependency

```
[(token.text, token.dep_, token.head.text) for token in doc]
```

係り受けを見ると、「何が何にかかっているか」を少し追えます。

これにより、

- 否定がどこにかかっているか
- 主張の中心述語は何か
- 誰に関する感情か

を少し見やすくなります。

5. memory_project で GiNZA が役立つ理由

5.1 topics / keywords の改善

今は token を雑に切って先頭数件を topics にしています。

GiNZA を入れると、

- 名詞だけを topic 候補にする
- 固有名詞を優先する
- 原形ベースで keyword を統一する

といった改善が可能です。

例えば、

「本屋で自分で本を選ぶのが面倒で、AIに一冊決めてほしい」

のような文から、

- 本屋
- 本
- AI
- 選ぶ
- 決める

のような、少し意味のある軸を取りやすくなります。

5.2 否定への対応

今のルールは「不安」「嫌い」「困る」が含まれるだけで反応しやすいです。

そのため、

- 不安はない
- 嫌いではない
- 困っていたわけではない

のような否定文を誤判定しやすいです。

GINZA があれば完全ではないにせよ、

- 否定補助語があるか
- 感情語が述語として使われているか

を見る土台ができます。

5.3 reason_codes の説明責任

GINZA を導入するとデバッグは少し難しくなります。

その代わり、reason_codes を

- topic_contains_proper_noun
- contains_negative_polarity
- has_person_entity
- has_future_time_expression

のように、もう少し中身のある根拠へ進化させやすくなります。

5.4 将来の検索基盤

embedding 検索をやる前でも、

- normalized text
- topic
- keyword
- entity

がまともに取れていると、検索と再想起の質が上がりやすいです。

GINZA はその基礎工事に向いています。

6. GINZA を入れてもできないこと

GiNZA は万能ではありません。ここを誤解すると設計を誤ります。

6.1 本当の感情理解

GiNZA は形態素解析・構文解析の土台です。

「本心ではどう感じているか」や「複雑な感情の揺れ」までは直接は分かりません。

6.2 深い意図理解

「この人は本当に何を望んでいるのか」までを高精度に理解するには、
GiNZA だけでは足りません。ルール、履歴、必要なら LLM 補助が要ります。

6.3 長い文脈理解

GiNZA は主に文単位、局所文脈単位の解析が得意です。
複数ターンをまたぐ感情変化や、会話全体の物語理解は別レイヤーの仕事です。

6.4 比喩・皮肉の理解

「それ最高だね」と言いつつ実は皮肉、のようなものは GiNZA だけでは難しいです。

したがって、GiNZA を入れる目的は

- 記憶判定を魔法のように賢くすること

ではなく、

- ルールを書きやすくし
- topic / keyword / entity / 否定検出の土台を改善すること

に置くべきです。

7. 他ライブラリとの比較

7.1 SudachiPy

SudachiPy は形態素解析器として強いですが、単体では spaCy のような統一 API や doc 構造を持ちません。

memory_project では token だけでなく、pos, lemma, entity, dependency をまとめて扱いたいので、GiNZA の方が相性がよいです。

7.2 Janome

Janome は軽くて導入しやすいですが、解析精度や拡張性では GiNZA より弱くなりやすいです。プロトタイプの軽さだけを見るなら候補ですが、今後の拡張を考えると GiNZA の方が自然です。

7.3 fugashi / MeCab

fugashi や MeCab は token 化の土台として優秀です。

ただし `memory_project` が欲しいのは token 化だけではなく、

- spaCy 互換の doc
- lemma
- entity
- dependency

まで含めた一貫した解析です。

GiNZA はこの一貫性が強いです。

7.4 素の spaCy 日本語モデル

spaCy 単体でも日本語処理はできますが、日本語向けの解析パイプラインとして GiNZA の方が扱いやすい場面が多いです。

特に、日本語でルールベース解析を積み上げたいプロジェクトでは GiNZA を選ぶ理由になります。

8. なぜ `memory_project` では GiNZA を選ぶのか

一言で言うと、

- 日本語中心
- 会話文中心
- 後で topic / keyword / entity / recall を育てたい
- ルールベース解析をまだ維持したい

という条件に合っているからです。

GiNZA を選ぶ理由は、最高精度だからではありません。

今の設計と次の拡張計画に対して、バランスがいいからです。

GiNZA を選ぶ主な理由

- spaCy API に乗るので Python 側で扱いやすい
- token, lemma, pos, entity, dependency を一貫して扱える
- 既に依存として導入済み
- 将来の検索と想起改善に繋げやすい
- 日本語ルールベース解析の足場として十分強い

GiNZA を選ばない理由になりうるもの

- とにかく軽さだけを優先する
- token 化だけであれば十分
- 日本語解析を最小の依存で済ませたい

この場合は SudachiPy 単体や別ライブラリの方が軽いことがあります。

9. memory_project への導入方針

一気に全面導入するのは避けるべきです。

順番は次のように切るのが堅いです。

段階1

topics と keywords だけ GiNZA 化する

- token を GiNZA で生成
- 名詞・固有名詞から topics
- 原形ベースで keywords

段階2

reason_codes を少し改善する

- 名詞密度
- 固有表現
- 否定の検出
- 未来表現の位置

段階3

facets を改善する

- 関係性
- タスク
- decision_support
- emotional friction

段階4

将来の embedding 検索や recall ranking へ接続する

10. まとめ

GINZA は、memory_project を一気に賢くする魔法ではありません。

しかし、今の正規表現ベース解析を、少なくとも「日本語の文を少し丁寧に扱う解析器」へ押し上げる力があります。

特にこのプロジェクトでは、

- topic / keyword の改善
- reason_codes の説明責任向上
- 否定や固有表現への足場
- 将来の検索拡張

という点で導入価値があります。

最初の導入点は、text_analyzer.py の token / topic / keyword 周りだけで十分です。

そこから段階的にルールを寄せていくのが、実装コストと説明責任の両方にとって良い進め方です。